

Towards Cardinality-Aware Local Search for SAT with Cardinality Constraints

Shuli Hu, Dian Ling, Jiaqi Li and Minghao Yin*

School of Information Science and Technology
Northeast Normal University, China

{husl903, lingd246, lijq515, ymh}@nenu.edu.cn

Bremen, Germany · 15-21 August, 2026



Outline

Background

Motivation

Method

Results



Background

Background

Motivation

Method

Results

Propositional Satisfiability (SAT)

Definitions

Given a propositional formula φ , test whether there is an assignment to the variables that makes φ true.

Example

$$\varphi = (x_1 \vee \neg x_2) \wedge (x_1 \vee x_2 \vee x_3) \\ \wedge (x_2 \vee \neg x_4) \wedge (\neg x_1 \vee \neg x_3 \vee x_4)$$

One satisfying assignment is

$$\alpha = (x_1, x_2, x_3, x_4) = (1, 1, 0, 1)$$

- SAT is the first problem proved NP-complete [Cook, STOC'71]
- A core problem in computer science and a basic problem in logic

A cardinality constraint counts how many literals are true

General Form

$$\sum_{i=1}^r l_i \bowtie k, \quad \bowtie \in \{\leq, =, \geq\}$$

At Least k

$$x_1 + x_2 + x_3 \geq 2$$

At Most k

$$x_1 + x_2 + x_3 \leq 1$$

Exactly

$$x_1 + x_2 + x_3 = 1$$

Normalization

At-most and equality constraints can be rewritten as At-Least- k constraints.

A CNF clause is the $k = 1$ case. KNF generalizes it to **klauses** with arbitrary k . [Reeves et al., CAV'24]



Motivation

Background

Motivation

Method

Results

Real models need logic and counting

Dependency: $x_A \rightarrow x_B$ becomes $\neg x_A \vee x_B$

Counting: $x_A + x_B + x_C \leq 2$

Planning

Clauses encode preconditions; counts limit simultaneous choices.
[Rintanen et al., AIJ'06]

Scheduling

Clauses encode precedence; counts enforce resource capacity. [Horbach, Ann. Oper. Res.'10]

Verification

Clauses encode logic; counts express bounded resources or selections.
[Vizel et al., Proc. IEEE'15]

The input is neither “only clauses” nor “only arithmetic”: it has a large Boolean core plus a smaller set of high-level counting requirements.

CNF encoding hides the original counting structure

Before: one explicit At-Least- K relation

After: many auxiliary variables and clauses

Why?

- Reuse highly optimized SAT solvers.
- Preserve satisfiability.
- Benefit from mature propagation and learning.

The actual cost

- **Lost:** structure information.
- **Added:** encoding overhead. [Wynn, arXiv'18]
- **Ordering:** strongly affects performance. [Reeves et al., AAAI'25]

Direct cardinality solvers

Pseudo-Boolean Solvers

Native reasoning over linear Boolean constraints.

Remaining limitation

The paper reports weak practical performance when most input constraints are already standard CNF. [Elffers & Nordström, AAAI'20]

Native SAT Solver

CDCL with native propagation on cardinality constraints. [Reeves et al., CAV'24]

Remaining limitation

It could not get better performance on large and long cardinality constraints.

Local search is an efficient heuristic method

Search Space

$$\mathcal{S} = \{0, 1\}^n$$

Every Boolean assignment α is a candidate solution.

Neighborhood

$$\mathcal{N}(\alpha)$$

An operator generates nearby assignments.

Scoring Function

$$\text{score}(\alpha')$$

Rank the neighbors and select the next move.

Start: choose an assignment α_0

Iterate: generate $\rightarrow$ score $\rightarrow$ move

If every constraint is satisfied,
return **SAT** with the assignment.

At the cutoff, return **Unknown**:
local search cannot prove UNSAT.

What's the issue of ordinary LS?

The scoring function significantly influences the search direction

Balance

We cannot evaluate the progress of cardinality constraints with only make-break value.

Let's consider this example:

Formula

$$x_1 + x_3 + x_4 + x_5 + x_6 \geq 3 \wedge (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_4) \wedge (x_2 \vee x_5) \wedge (x_3 \vee x_6)$$

Assignment

$$\alpha = (0, 0, 0, 1, 0, 0)$$

What's the issue of ordinary LS?

The scoring function significantly influences the search direction

Balance

We cannot evaluate the progress of cardinality constraints with only make-break value.

Let's consider this example:

Formula

$$x_1 + x_3 + x_4 + x_5 + x_6 \geq 3 \wedge (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_4) \wedge (x_2 \vee x_5) \wedge (x_3 \vee x_6)$$

Assignment

$$\alpha = (0, 0, 0, 1, 0, 0)$$

Make-Break: $\text{score}(x_2) = 2$, $\text{score}(x_3) = 2$ However, flipping x_3 is better than x_2 .

Our Contributions

KNF Preprocessing

Simplify without hiding the cardinality structure.

Cardinality-aware scoring

Measure both clause gain and cardinality progress.

History-aware initialization

Reuse stable phases across restarts.

Adaptive flip-swap search

Use swap moves to escape flip-based stagnation.

Our Contributions

KNF Preprocessing

Simplify without hiding the cardinality structure.

Cardinality-aware scoring

Measure both clause gain and cardinality progress.

History-aware initialization

Reuse stable phases across restarts.

Adaptive flip-swap search

Use swap moves to escape flip-based stagnation.

Performance: 557 / 814 solved with lowest PAR-2



Method

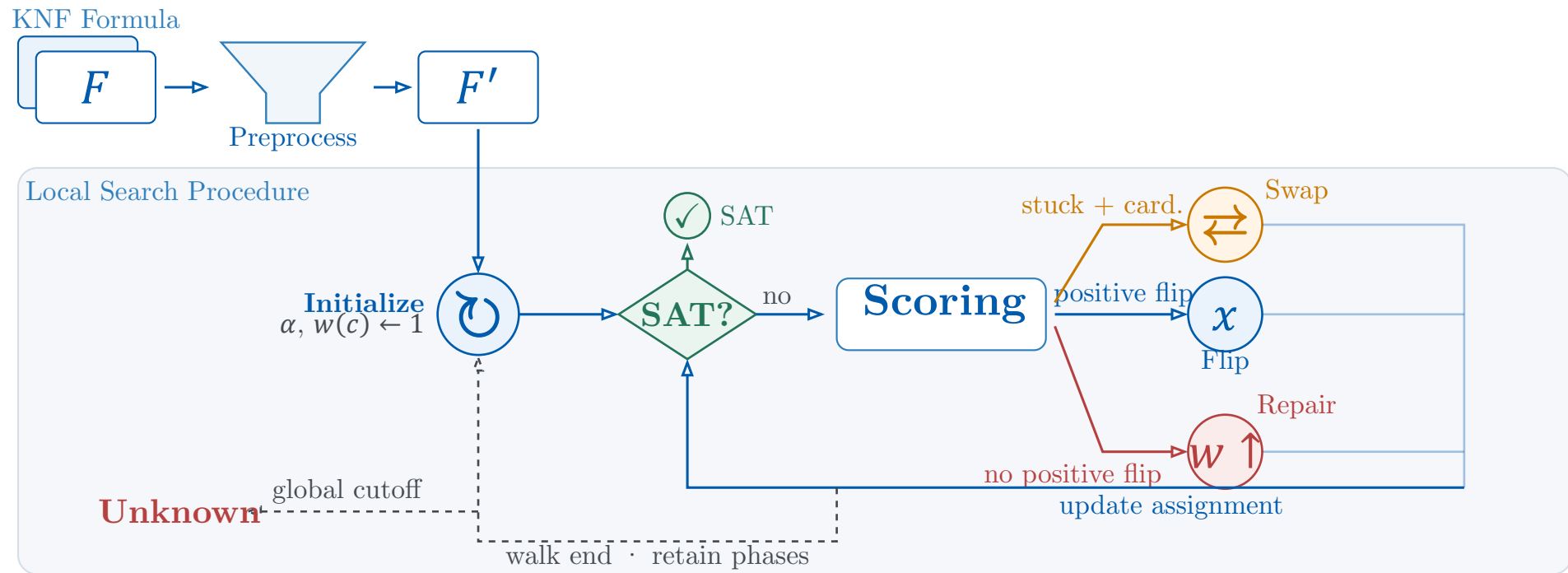
Background

Motivation

Method

Results

CardSAT-LS Framework



Preprocessing simplifies the formula without flattening KNF

Generalized UP

$$c : \sum_{i=1}^r l_i \geq k, \quad k = |c|$$

Every literal in c must be true.

Assigned literals are removed, and the bound is decremented when a true literal is consumed.

Generalized Resolution Check

$$y + \sum_i x_i \geq k_1 \otimes_y \neg y + \sum_j z_j \geq k_2$$

$$\sum_i x_i + \sum_j z_j \geq k_1 + k_2 - 1$$

Applied only when supports are disjoint and a bounded elimination check is cheap.

Additional preprocessing follows PRS. [Chen et al., SAT Competition'22]

The score rewards clause progress

$$\text{score}(x) = \text{score}_{m-b}(x) + \text{score}_{\text{card}}(x)$$
$$\text{viol}(c) = \max(0, k - \sum_i l_i)$$

For Clauses

Weighted clauses satisfied minus weighted clauses broken by flipping x .

For Cardinality

Weighted decrease in violation across the cardinality constraints containing x . [Lei et al., SAT'21]

The tie of earlier example becomes $\text{score}(x_2) = 2$ and $\text{score}(x_3) = 3$, so the solver selects the move that improves the count.

Restarts reuse stable phases

**First Start
Random
assignment**

Avoid a fixed initial bias.

**During the Search
Record last flips**

An early last flip suggests that the current phase is stable.

**Restarting
Reuse & Reset**

Keep stable phases / reset to 0.

$\text{timestamp}[x] \leq \text{thres}_{\text{bb}}$: reuse the saved phase.

$\text{timestamp}[x] > \text{thres}_{\text{bb}}$: reset the variable to 0.

We call it **Fake-backbone Initialization**.

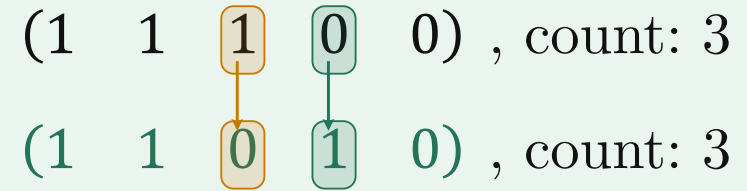
Swap & Flip

Search Policy

- (1) If a positive-score variable exists, flip the best one.
- (2) If no improving move exists, increase weights on unsatisfied clauses.
- (3) After prolonged stagnation, try a swap with probability p_{swap} .

Example

(1 1 1 0 0) , count: 3
(1 1 0 1 0) , count: 3



Flipping one true and one false variable preserves the cardinality constraint while reaching a neighborhood unavailable to a single flip.



Results

Background

Motivation

Method

Results

Evaluation covers complete and local-search baselines

814

known-SAT

instances in main comparison

Complete solvers × 3

CCDCL: KNF SAT

Open-WBO · MaxSAT

RoundingSAT · PB

5

benchmark sets

competition and applications

SAT LS × 4

CCAnr, TaSSAT

WalkSATlm · YalSAT

3600 s

cutoff per run

30 GB memory

PB LS × 3

DLS-PBO, NuPBO

NuPBO-DeepOpt+

Ours: CardSAT-LS KNF-native, Cardinality-aware, Local search

Solved counts by benchmark

Family	Solver	MaxSAT24	SAT25	DES	MVC	WSNO	Total
		368	191	120	117	18	814
Exact	CCDCL	258	102	106	41	18	525
	Open-WBO	249	70	116	41	17	493
	RoundingSAT	214	66	108	51	18	457
SAT LS	CCAnr	117	46	109	39	12	323
	TaSSAT	153	61	120	78	0	412
	WalkSATlm	42	29	0	33	0	104
	YalSAT	86	37	35	35	13	206
PB LS	DLS-PBO	243	25	119	73	18	478
	NuPBO	239	38	108	77	18	480
	NuPBO-DeepOpt+	206	24	106	75	18	429
Ours	CardSAT-LS	291	46	120	82	18	557

557 / 814 solved

+32 vs CCDCL · best or tied-best on 4 / 5 sets

CardSAT-LS leads on solved count and PAR-2

557 / 814

instances solved

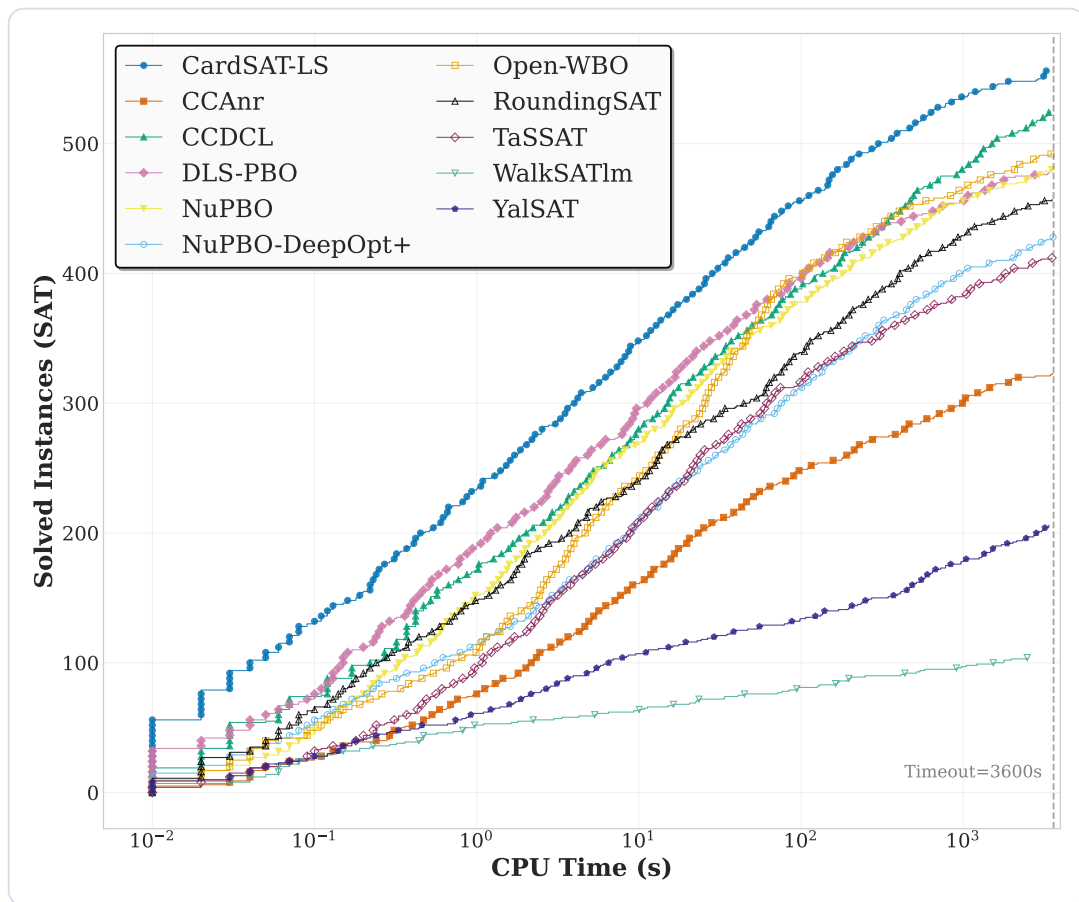
32 more than CCDCL (525)

2369.06

PAR-2

12.5% lower than CCDCL (2708.11)

Among the 11 evaluated solvers, CardSAT-LS achieves the highest solved count and lowest aggregate PAR-2.



Preprocessing is the strongest isolated component

Variant	Enabled	#Solved / PAR-2
+P+B+S	Preprocess · Backbone · Swap	557 / 2369.06
P only	Preprocessing	549 / 2453.64
B only	Fake-backbone initialization	521 / 2682.85
S only	Swap operator	527 / 2673.94
None	None of P, B, or S	486 / 2999.67

Every isolated component beats the 486-solve base. Preprocessing is strongest alone at 549, while full P+B+S reaches 557.

Hybrid CCDCL-LS covers 159 more instances

Hybrid: CardSAT-LS +
CCDCL

CardSAT-LS proposes phases
during each walk; CCDCL
retains complete reasoning.

CCDCL

589 / 896

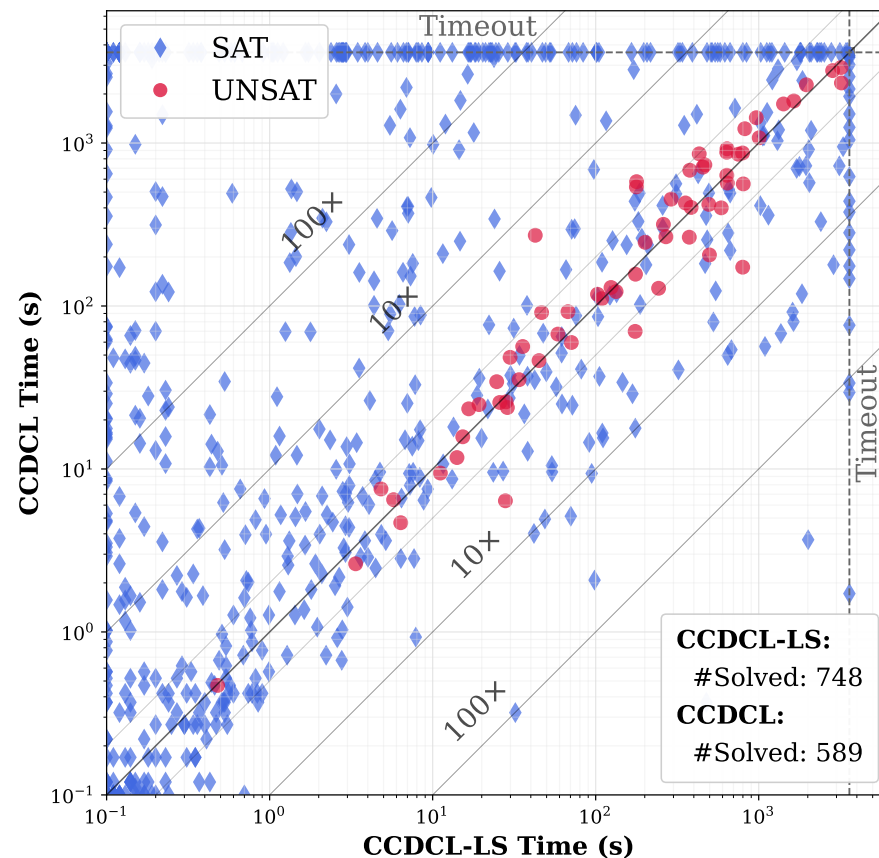


CCDCL-LS

748 / 896

+159 solved

+27.0% relative to CCDCL.



Conclusions & Future Work

CardSAT-LS is a first step toward local search that reasons directly over mixed CNF and cardinality constraints.

Heuristic

Search more efficiently

Develop stronger heuristic strategies for cardinality-aware local search.

Instance Features

Understand when it helps

Relate structural features to solver performance across different SAT-with-cardinality families.

github.com/virgiling/CardSAT-LS

Thank You!

Contact: virgiling7@gmail.com